Assignment 3: Submission & Reflection

MLOps & Infrastructure for Machine Learning (Term 3), IIT Madras

Student Name: Girinath Bhatt S

Roll Number: DA25M511

Submission Links

- **GitHub Repository:** https://github.com/girinathbhatts/mlops-pytorch-pipeline
- **Final PR (Release to main):** https://github.com/girinathbhatts/mlops-pytorch-pipeline/pull/9
- **Validation Evidence PR:** https://github.com/girinathbhatts/mlops-pytorch-pipeline/pull/8

Reflection: What was the most challenging part?

Building an end-to-end MLOps pipeline that spans local PyTorch development, multi-stage Docker containerization, and Kubernetes orchestration involved a few tricky integration issues. The most challenging aspect was coordinating state and storage across decoupled container lifecycles in Kubernetes while managing local cluster networking and image resolution.

1. Decoupled Lifecycle and PVC Race Conditions

In Kubernetes, training and serving are decoupled into a batch Job and a long-running Deployment. The primary challenge was ensuring that the serving Deployment did not crash or enter a CrashLoopBackOff state before the training Job finished writing the model checkpoint (classifier_v1.pt) to the shared PersistentVolumeClaim (checkpoint-pvc). Initially, deploying all manifests simultaneously caused the FastAPI application to start without a valid model weights file, resulting in 503 HTTP responses on health checks. To solve this, I decoupled the rollout into a sequenced workflow using 'kubectl wait --for=condition=complete job/pytorch-training' before triggering the serving Deployment, while configuring proper readinessProbe with initialDelaySeconds: 15 and livenessProbe in the serving manifest to ensure traffic is only routed after PyTorch loads the model into memory.

2. Local Container Image Resolution in Minikube

Another subtle challenge was Minikube's Docker daemon isolation. When building images locally on the host via Docker Desktop (mlops-train:v1 and mlops-serve:v1), Minikube could not resolve them by default and attempted to pull from Docker Hub, resulting in ImagePullBackOff errors. Setting imagePullPolicy: Never and explicitly loading the images into the Minikube cache via 'minikube image load <image-name>' resolved this without needing a remote container registry.

3. Resource Constraints and Metrics Server for Autoscaling

Configuring the Horizontal Pod Autoscaler (HPA) highlighted the dependency on cluster-level metrics infrastructure. In a minimal Minikube environment, without metrics-server enabled and configured with --kubelet-insecure-tls, the HPA reports <unknown> for CPU utilization targets. This showed the difference between defining scaling rules in YAML and actually having the cluster metrics available to trigger them.

Summary

This project reinforced the importance of declarative infrastructure, persistent storage lifecycle management, configuring readiness and liveness probes properly, and separating training from inference workflows in MLOps.